

Compact Fixed Controller S Computation from a 38 Phase Cyclic Tag System

7 October 2026

Abstract

A fixed finite controller can select native S-combinator contractions that simulate a compact universal cyclic tag system. We give a concrete 38-phase instance, an explicit compiler from finite natural-register machines, and a direct compiler from conventional integer-tape Turing machines with arbitrary finite alphabets. A fixed regular tree language recognizes source halting. At its first accepted sample, a fixed current-tree decoder returns the exact register-machine result. The controller restarts at the root on each invocation, retains no state between invocations, and has an all-input linear invocation bound. The generic cyclic-tag-to-S construction is due to Cinematic Strawberry, whose earlier 912-phase construction already establishes fixed-controller universality for Boolean tapes. The contributions here are the compact source route, its explicit formal compiler, and the generated-origin and first-event numerical composition for this instance. All 67 project modules passed fresh Lean 4.33.1 kernel replay. Complete operation-cost interpretations are stated separately from the checked semantic and size lemmas.

1 Introduction

The sole rewrite rule of the S combinator is

$$S x y z \longrightarrow_S x z (y z),$$

with application associated to the left. The computational question depends on the encoding, reduction strategy and observation interface. This paper studies a selected trajectory of finite closed S trees, with source halting recognized during continuing reduction. The trajectory is generated by one fixed finite root-reset controller. All auxiliary interfaces are specified before the source machine and input are supplied.

Cinematic Strawberry’s prior construction [1] establishes this kind of computation through a fixed 912-phase cyclic tag system. Its public headline already includes regular-language halting on the actual root-reset trajectory, conventional Boolean infinite-tape semantics, and exact source-row and terminal-bit readout [2, 3, 4]. We use its generic cyclic-tag-to-S encoding, scheduler and finite selector. The present result is a different compact instantiation and an explicit proof of its source, intermediate-event and numerical-output obligations.

The source endpoint is ais523’s 19-symbol alternating tag system UT19 [7]. A one-hot translation gives a 38-phase binary cyclic tag system. An explicit compiler supplies a universal finite-register source with an exact numerical output convention. A further checked compiler begins with a conventional integer-indexed tape and an arbitrary finite alphabet. The main proof tracks all intermediate S samples: a completed phase-17 pattern can be copied into several parts of a tree, so checkpoint correspondence alone does not establish that the first matching occurrence carries the intended result. Generated label and audit invariants prove that every match in the first accepted tree has the same frozen payload.

The contribution is therefore threefold. First, the fixed cyclic tag endpoint has period 38 with an explicit machine/input compiler. Second, the first-event result theorem covers every finite natural-register program and input, with no residual source-event, liveness or witness-correctness hypothesis. Third, the composition has a reproducible kernel-checked

implementation, including a direct finite-alphabet tape bridge. The change from 912 to 38 is a change in one construction parameter; it implies no corresponding factor of improvement in runtime, control-state count or term size.

2 The computational model

Definition 2.1 (Terms and native steps). The set $\mathcal{T}_S$ consists of finite terms generated by S and binary application. Its occurrence size is $|S| = 1$ and $|uv| = 1 + |u| + |v|$. A native step contracts exactly one occurrence of $Sxyz$ to $xz(yz)$ inside an arbitrary application context.

Sizes count occurrences, including every repeated subtree. A compressed DAG implementation may use fewer distinct nodes; all asymptotic readout statements in this paper use occurrence-tree size.

Definition 2.2 (Root-reset finite selector). A controller has a finite control set and an occurrence cursor on the current tree. A cursor is a focus with its parent-frame path. A transition observes only the current control, node kind and incoming side. Primitive commands change control, move along one child or parent edge, contract a focused native S redex, or reject. Each invocation starts in the same control at the root. Before the selected contraction, navigation is read-only. Between invocations only the resulting bare S term is retained.

The cursor may have unbounded depth as the input tree grows; finiteness concerns control, not tree size. The imported selector contract makes this distinction explicit. Its inter-invocation state is literally `Unit`. For the fixed controller C used below there is a constant a such that every invocation on an arbitrary finite tree T stops within $a(|T| + 1)$ primitive microticks. On a normal tree it makes no contraction. On every encoded trajectory in the theorem it selects exactly one native contraction and returns its successor. The constant a , controller and observation algorithms are independent of the source machine and input. The bound is per invocation, not per simulated computation.

Write $F_C : \mathcal{T}_S \rightarrow \text{Option}(\mathcal{T}_S)$ for the selected successor. For an encoded source instance z , define $T_z(0) = E(z)$ and $F_C(T_z(i)) = \text{some}(T_z(i+1))$. The proof establishes this equation for every i , so the totalization used in the implementation is immaterial on encoded paths. Observation occurs at each resulting tree, including the initial tree. For a fixed Boolean observer H , define

$$\text{First}_H(T, n) \iff H(T(n)) \wedge \forall j < n, \neg H(T(j)).$$

The decoder $D : \mathcal{T}_S \rightarrow \text{Option}(\mathbb{N})$ receives only the current tree. It has no source program, input, trace, checkpoint index or execution oracle as a runtime argument.

2.1 Source machines

A register machine has finitely many labels and $R \geq 1$ natural registers. Its instructions are increment and jump, decrement with separately specified positive and zero successors, and halt. A zero test leaves a zero register unchanged; a positive decrement subtracts one. Halting configurations are absorbing. For input vector u , let $M[u] \Downarrow v$ mean that a finite run reaches halt with register 0 equal to v .

The tape source has finite state and alphabet sets, a start state, a blank symbol, and a finite partial transition table. A transition writes at the current integer coordinate and moves by -1 , 0 or 1 . No transition means halt. An input word occupies coordinates $0, 1, \dots$, the head starts at 0 , and the rest of the tape is blank. States contain a genuine total tape function $\mathbb{Z} \rightarrow \Sigma$. There is no bounded visited-window hypothesis. Finite transition functions are supplied by explicit finite tables when measuring the encoding's binary description length.

3 Uniform computation theorems

Theorem 3.1 (Register results). *There are a fixed root-reset finite controller C , a fixed regular tree language H , an effective finite-tree encoder E_R , and a fixed total decoder D such that, for every finite register machine M , natural input vector u and $v \in \mathbb{N}$,*

$$M[u] \Downarrow v \iff \exists n (\text{First}_H(T_{M,u}, n) \wedge D(T_{M,u}(n)) = \text{some}(v)).$$

Every adjacent pair in $T_{M,u}$ is one native S step. Every invocation of C on every finite input tree satisfies the fixed linear bound above. The encoder has a singly exponential bound in an explicit binary source description; the decoder has a polynomial all-input bound in current occurrence-tree size.

The semantic equivalence is `SOnlyUniversality.result_iff_first_event`. The encoder and decoder are ordinary executable definitions. The construction of E_R has no halting-time parameter and never runs the source to determine its initial S term. The resource interpretation is made precise in Section 9.

Theorem 3.2 (Conventional tape halting). *There is an effective encoder E_T for every finite-alphabet tape machine M and finite word w , using the same C , H and D , such that*

$$M \text{ halts on } w \iff \exists n H(T_{M,w}(n)).$$

Equivalently, a first accepted sample exists. If the source does not halt, H rejects every finite sample. At the first accepted sample, D returns the code of the finite left stack in the specified two-stack simulation.

The halting endpoint is `SOnlyTuringUniversality.halting_iff_event`; the value-specific endpoint is `SOnlyTuringUniversality.left_stack_result_iff_first_event`. The left stack retains explicit blank cells and visited extent. Its code is an observable of this deterministic simulation history, rather than a canonical arbitrary final tape-output word. The general natural-result interface is supplied by Theorem 3.1.

4 Finite registers and the numerical BP2 frontend

The source reduction is an executable composition, with no supplied bound on source running time. Its intermediate mechanisms follow ais523’s Amnesiac From Minsk [10], Waterfall Model [9], and Brainpocalypse II [8] constructions; the Waterfall account also credits zzo38’s independent model. The explicit register-frontend tables, compiler composition, and proofs described here are this development’s contributions. The generic CTS-to- S encoding and root-reset controller are supplied by Cinematic Strawberry’s pinned construction.

A source machine has $R \geq 1$ natural registers, $Q \geq 1$ labels, an entry label, and instructions $\text{INC}(i, t)$, $\text{DECJZ}(i, t_+, t_0)$, and HALT . Zero testing leaves a zero register unchanged. BP2 instead executes a finite list of $+i, -i$: positive decrements advance normally, while a zero decrement sets that counter to one and restarts at command zero, retaining all other counters. Falling off the list halts; all BP2 counters initially equal zero.

Theorem 4.1 (Exact register frontend). *For every such M and input tuple a , the total function `SOnlyMachineNat.compile` produces a nonempty finite BP2 program $B(M, a)$ with dense counter labels. It halts exactly when $M(a)$ halts. For each $v \in \mathbb{N}$, it halts with first counter $2v + 3$ exactly when $M(a)$ halts with register zero equal to v .*

Construction and proof. First use positive amnesiac counters: $+i$ increments and follows a fixed successor p_i ; $-i$ decrements above one and follows s_i , or leaves one unchanged and follows f_i . Lean stores these values minus one. Keep $D_i = r_i + 1$ and allocate five helpers I_q, S_q, F_q, U_q, V_q at *every* label, initially one. Let $E(q)$ be $+I_q, +S_q$, or halt according to the source instruction. The successor table is

	p	s	f
D_i	$-I_0$	$-S_0$	$-F_0$
I_q	$+D_i$	$E(t)$	$-I_{q+1}$
S_q	$+F_q$	$-V_q$	$-S_{q+1}$
F_q	$-D_i$	$-U_q$	$-F_{q+1}$
U_q	$-F_0$	$E(t_+)$	$+V_q$
V_q	$-S_0$	$E(t_0)$	$+U_q$

Here instruction-dependent entries apply to the appropriate instruction; inapplicable instruction-dependent entries and end-of-chain fallbacks are halt. Scans visit all labels. A chain with exactly one marked value two passes the ones, restores the mark to one, and takes precisely its success edge. An increment marks I_q , increments D_i , and scans I . A decrement marks S_q, F_q , tests D_i , then uses the S, V, U, F or symmetric F, U, V, S route to restore every helper and select t_+ or t_0 . Thus the instruction-boundary invariant is exact, including shared data registers and repeated call sites (`SOnlyMachine.macro_simulation`).

For $k = R + 5Q$ amnesiac counters, introduce $4k + 1$ Waterfall clocks: data X_i , action clocks Q_a for $a = +i, -i, H$, and recovery clocks T_i . At normalized action (c, a) set $X_i = 2c_i$, $Q_a = 0$, and every other control clock to four. With e_Z a unit vector, the signed trigger rows are

$$\begin{aligned} b_{Q_{+i}} &= 2e_{X_i} + 4e_{Q_{+i}} - 4e_{Q_{p_i}}, & b_{Q_{-i}} &= -2e_{X_i} + 4e_{Q_{-i}} - 3e_{T_i}, \\ b_{X_i} &= 2e_{X_i} + 3e_{T_i} - 4e_{Q_{f_i}}, & b_{T_i} &= \mathbf{1} + 3e_{T_i} - 4e_{Q_{s_i}}. \end{aligned}$$

Use $A_Z = b_Z + 5 \cdot \mathbf{1}$ and $A_{Q_H} = 0$, adding coincident terms. All nonhalt entries lie in $[1, 9]$. Subtracting the common minimum after a trigger cancels its uniform offset. An increment reaches the next normalized action; a decrement makes $X_i = 2c_i - 2$ and $T_i = 1$. If $c_i = 1$, X_i is uniquely minimal and restores the failure state; otherwise T_i is uniquely minimal and its recovery row restores the successful-decrement state. These arithmetic identities prove tie-free simulation (`SOnlyMachineClock.step_simulation`).

BP2 implements each clock with a value cell v_i and pending marker z_i , plus one marker m . For a nonnegative vector b , write $\text{Add}(b)$ and $\text{Sub}(b)$ for literal unary increments and decrements. The initializer is

$$\text{Add}(a); -m; +m; \text{Sub}(a), \quad a(v_i) = w_i, \quad a(z_i) = 1, \quad a(m) = 0.$$

Its first zero guard preserves initialization across a restart; subsequent visits cancel exactly, even when working cells are zero. Trigger i uses the same pattern guarded by z_i , adding $A_i - e_i$ to value cells. A sweep subtracts one from every v_i ; tests $-z_i; -v_i; +v_i; +z_i$ mark the unique zero and restart. The selected trigger then compensates for the test's automatic increment and applies exactly A_i . The terminal pair $-m; -m$ restarts ordinary sweeps. The halt trigger instead adds $2 \cdot \mathbf{1} - e_h$ and increments m ; one final sweep falls off with $v = y + \mathbf{1}$, every $z_i = 1$, and $m = 0$. Hence the first value is $2(r_0 + 1) + 1 = 2r_0 + 3$. Finite positive macro durations and absorbing halt states give reflection at arbitrary target times, not only checkpoints. Dense renumbering preserves the output coordinate. These are `SOnlyMachineNat.halt_iff` and `SOnlyMachineCorrectness.output_iff`. $\square$

The formal compiler initializes clocks at $V + 5$, whereas the earlier selective Python construction uses $V + 1$ and allocates helpers only where needed. The formal count is

$c = 8k + 3$, $k = R + 5Q$; its checked bound, with $N_c = 4k + 1$ and $s = \sum_i a_i$, is

$$L \leq 2c(2s + 9) + N_c(18c + 2) + c + 4N_c + 4 \quad (\text{SOnlyMachineBounds.compile_length_bound}).$$

The two variants preserve semantics but are not byte-for-byte identical.

5 Programmed UT19 execution and its first event

The next reduction uses ais523's CC0 UT19 construction [7]. Its literal productions, in published one-based notation, are

$$\begin{array}{lll} 1 \mapsto (2, 3) & 2 \mapsto (4, 4) & 3 \mapsto (18, 4) \\ 4 \mapsto (1, 1, 19) & 5 \mapsto (7, 9) & 6 \mapsto (8, 9) \\ 7 \mapsto (10, 10) & 8 \mapsto (11, 10) & 9 \mapsto (18, 10) \\ 10 \mapsto (5, 6) & 11 \mapsto (6, 5, 19) & 12 \mapsto (14, 14, 14, 14) \\ 13 \mapsto (15) & 14 \mapsto (16, 16) & 15 \mapsto (16, 17) \\ 16 \mapsto (12, 13) & 17 \mapsto (12, 12, 12, 12) & 18 \mapsto (18) \\ 19 \mapsto \epsilon \end{array}$$

Word powers denote literal repetition. A microstep removes one symbol, appends its production only in *take* phase, and toggles take/skip. The designated event is a *prestate* with take phase and head 18; a skipped 18 is not an event.

For a BP2 program of length L and c counters, add a halt counter $H = c + 1$. Choose the least power of two $N \geq L + 3$ and put $W = 2N$. The seed is

$$19 M_1 K_1 \cdots M_c K_c M_H K_H M_{H+1}, \quad K_i = (12, 13)^2.$$

Each memory has W cells. The expanded program is dummy, the L commands, halt-A, halt-B, and padding to N slots. Every slot uses two halfcommands; each halfcommand comprises Command, Parity, and Reset passes, where a pass consumes exactly its entrance word.

Lemma 5.1 (Finite programmed-memory window). *For any prescribed W with parities there is an explicit initializer giving those observations at times $0, \dots, W - 1$, independently of the incoming Command alignment history. Odd Reset restores this initializer exactly.*

Proof sketch. A cell with dynamic bit h_i is (5, 6) or (6, 5, 19); a fixed inverter bit $a_i = 1$ adds (1, 1, 19). Put $c_i = h_i \oplus a_i$. Direct production calculation gives, under normal Parity and Reset, $c' = Pc \oplus g\mathbf{1}$, where P is inclusive prefix XOR and $g = 1$ means skip alignment. Width is observed *before* updating: $v_t = \bigoplus_i c_{t,i}$. For $W = 2^d$, the unforced observation transform is

$$v_t = \bigoplus_{i: (i \& t) = t} a_i, \quad a_i = \bigoplus_{j: (j \& i) = i} v_j.$$

The superset transform is its own inverse over $\mathbb{F}_2$. The formal proof uses even/odd index decomposition and identifies its recursive transform with the increasing-stride butterfly. The extensionally equal increasing-stride implementation uses $d = \log_2 W$ stages and $W \log_2 W / 2$ bit XORs. This count concerns that implementation; the output-equivalence theorem is not an exact operation-count theorem for direct evaluation of the recursive Lean definition. An alignment disturbance in update t first affects observation $t + W$; reset sets every h_i to zero. These facts are SOnlyTemporalMemory.forced_initializer_correct, initializer_eq_increasing_strides, and SOnlyMemory.memory_reset_vector. The emitted choice $d = \lfloor \log_2(L + 2) \rfloor + 2$ satisfies $2L + 3 < W \leq 4L + 8$ by SOnlySimulation.compiler_capacity and compiler_width_bound. $\square$

Memory widths are adjacent XOR differences of the desired component alignments, so prefix XOR telescopes to exactly those alignments. `SOnlySimulation.scheduledWidth_eq_published` checks equality with the literal width recipes, including unused halt-B and padding slots. For $\mathcal{C}(n) = (12, 13)^{2^n}$, ordinary BP2 value x is $\mathcal{C}(2x+1)$. A halfcommand increments n when Command starts take; a positive-exponent decrement subtracts one under even Reset. Thus $+i$ increments all exponents, then increments i and decrements the others. Similarly, $-i$ decrements all, then decrements only i and increments the others. These implement changes of two in the target exponent and zero elsewhere.

The zero branch requires more than ordinary decrement arithmetic. Decrementing $\mathcal{C}(0)$ produces a singleton 15 in Parity. At a source zero decrement it is selected, creates odd Reset, and leaves the protected word $D = (12, 12, 12, 12)$. Every other counter is incrementing in that halfcommand: no positive decrement is exposed to odd Reset, which would otherwise produce the wrong protected form. All memories reset exactly; any 18 they emit is skipped. The restarted dummy decrements all and then increments all, but D responds to the first half by becoming $\mathcal{C}(2)$, then $\mathcal{C}(3)$. The target therefore becomes one, as BP2 requires, while other values are retained. The initial leading 19 supplies the same skip alignment for the first dummy. This proves closure through arbitrarily many restarts (`SOnlySimulation.source_zero_decrement_run`).

At command i , memory time is $2i + 2$. Each epoch either resets or reaches halt-A's last needed observation $2L + 3 < W$, so the memory lemma is never used past its protected window. Halt-A decrements all exponents, then increments ordinary counters and decrements the halt counter with odd Parity. The latter's singleton 15 is skipped and disappears; Reset begins take with head 18. Exactly five passes after halt-A entrance, the event queue is

$$R_1 16^{4^{x_1+1}} \dots R_c 16^{4^{x_c+1}} R_H T,$$

where all separators are nonempty and contain no 16. The x_i are the final BP2 values. Thus maximal 16-runs give precisely the retained counters (`SOnlySimulation.finalEvent_readout`); repeated division by four recovers them. The first counter then yields $(x_1 - 3)/2$ for register output (`finalEvent_readMachineValue`).

Each preceding simulated step is a finite nonempty, event-free run. Unconsumed entrance symbols prevent exhaustion inside a pass, and memories supply nonempty output at its end. Positive simulation durations make nonhalting checkpoints cofinal, excluding an event at *every* finite microstep. Consequently `SOnlySimulation.source_halting_iff_event` is a two-way unbounded theorem, and `first_event_exact` identifies the actual first event queue, rather than only an expected macro-boundary.

6 The fixed 38-phase interface and conventional tapes

Encode symbol s by $\eta(s) = 0^{s-1} 1 0^{19-s}$. The fixed CTS has the nineteen encoded productions followed by nineteen empty appendants. Take and skip boundaries are phases zero and nineteen.

Lemma 6.1 (Exact block and event transfer). *Nineteen CTS steps simulate one UT19 microstep. At every CTS time t , the phase-17/head-one event holds exactly when UT19 is selected at microstep $\lfloor t/19 \rfloor$ and $t \bmod 19 = 17$. A first UT19 event at n therefore becomes the first CTS event at $19n + 17$.*

Proof sketch. After consuming a prefix, FIFO order leaves its unconsumed suffix ahead of all generated appendants. Scanning one 19-bit block appends exactly its production in the take half, or nothing in the skip half, and toggles the boundary phase. The remaining original bits

prevent premature exhaustion. The only one in $\eta(s)$ is at offset $s - 1$; phase 17 can therefore signal only selected symbol 18. Division with remainder covers arbitrary times. These are `SOnlySource.trajectory_simulation`, `event_at_arbitrary_time`, and `first_event_exact`. $\square$

At the event, the current CTS word is $10\eta(\text{tail})$, not a block-aligned encoding. Restoring the known seventeen-zero prefix reconstructs the whole UT19 event word; `SOnlyResultBits.decodeEventWord_correct` proves this structural readout. Composition gives `SOnlySimulation.compiled_halting_iff_cts_event` and `compiled_noPrematureEmpty`, the exact source interfaces consumed by the fixed S backend. In particular, UT19's rule $18 \mapsto 18$ is an event marker, not a claim that the tag queue or S term normalizes.

Finally, the source need not be assumed universal by convention. `SOnlyTapeStacks` treats a deterministic finite-alphabet machine on an integer-indexed tape, with left, stay, and right moves and absent transition meaning halt. Finite control stores the state and scanned symbol; nearest-first left and right stacks represent every remaining tape cell, with empty pop returning blank. For alphabet size A , use base $B = A + 1$ and

$$\text{code}([]) = 0, \quad \text{code}(a :: u) = (a + 1) + B \text{code}(u).$$

Nonzero digits preserve explicit blanks and make this code injective. Literal INC/DECJZ macros implement push by multiplication and addition, and pop by a finite-control residue cycle and quotient transfer. Registers zero and one hold the stacks; register two is scratch, restored to zero. The assembled table has $|Q_{\text{TM}}|A(5(A + 1) + 5) + 1$ labels and depends only on the machine; its entry selects the input's first symbol and the right register contains the remaining input. Positive-duration macro execution and cofinal checkpoints reflect halting even inside target macros (`SOnlyTapeCompiler.input_halting_iff`).

Hence `SOnlyTuringUniversality.halting_iff_event` covers arbitrary finite inputs without a tape-window or runtime premise. Arbitrary register programs retain their exact natural output. For this TM bridge the output is specifically the finite left-stack code, including retained blanks and visited extent: `left_stack_result_iff_first_event`. It is a history-dependent observable of this simulation, not a canonical final tape-output word.

7 The fixed S implementation

Let P be the literal 38-phase cyclic tag program obtained above. Its true-bit appendants contain 760 bits in total, of which 40 are ones. Fix its canonical balanced action dispatcher A . There are 76 phase/bit leaves. The leaf for phase 17 and bit true has route 0100011, with left and right encoded by 0 and 1. Define the S generator and selector by the pinned upstream operations:

$$\begin{aligned} E_P(b) &= \text{generator}(\text{compileActions}(P, A), b), \\ F_P &= \text{RootResetFiniteAllInputsTraceAgreement.selector}(P). \end{aligned}$$

Here b is the compiled initial bitword. P , A , the compiled action code and F_P are fixed. Source-specific information is confined to b .

Proposition 7.1 (Imported generic realization). *For every finite bitword b and every sample index i , the root-reset selector returns the next term of the generic persistent CTS simulation. That successor is obtained by one native S contraction. Public checkpoints decode to the corresponding iterates of P . The actual finite controller realizes this same selector on every input tree and satisfies its all-input stopping bound.*

Proof. Use the following pinned generic results, specialized at P :

- `RootResetFiniteAllInputsTraceAgreement.agreesOnEverySample;`
- `termOnlyPath_eq_persistentPath;`
- `RootResetFinitePrioritySelector.selectorContract.`

The local specialization `SOnly38.selector_eq_projected` identifies the bare-term selector with the invocation contract, and `SOnly38.trajectory_selects` and `trajectory_contracts` identify every sample, not just checkpoints. The generic dependency closure is attributed to Cinematic Strawberry [1, 5] and replayed without replacing its construction by a Python implementation. $\square$

7.1 A fixed completed-response pattern

The designated observation is a completed, fresh Local response shell for label $(17, \text{true})$. The shell contains the fixed dispatch route and nineteen completed appender-history arguments. Call its independent-hole pattern F_{17} . The language H consists of finite S trees containing an occurrence matching F_{17} . Holes place no equality constraint on distinct subtrees. Equal-variable matching is not silently used to obtain regularity.

Lemma 7.2 (Finite observer). *H is recognized by a fixed bottom-up tree automaton on every finite tree.*

Proof. Store one Boolean match bit for each occurrence in the fixed pattern, and one bit recording a match at any descendant. At an S leaf, a hole and an S pattern match, while an application pattern fails. At an application node, the bit for an application subpattern is the conjunction of its left and right child match bits. A hole always matches. The descendant bit is the disjunction of the root match and the child descendant bits. Structural induction proves agreement with pattern matching and descendant occurrence. The complete Boolean cube has $2^{|F_{17}|+1}$ states, including unreachable states, giving an explicit finite cover. This is checked in `SOnlyObserver`, in particular `event_finite_cover` and its transition-agreement lemmas. $\square$

The size of this uncompressed cover is a fixed construction constant, not a claim of an efficient minimal automaton. A separate implementation can share equal subpatterns; the theorem uses the occurrence-based cover above.

8 Global first-event provenance

Generic checkpoint simulation does not by itself identify the earliest occurrence of the newly chosen pattern. A native S contraction copies a subtree. Completed shells may survive in inactive fields or in frozen audits, and a response is assembled over many contractions. Correctness therefore requires an invariant covering every relevant descendant at every intermediate sample.

8.1 Licenses for generated shells

A proof-only license is an allowed pair of source label and frozen snapshot value for a fresh completed Local. The invariant is extensional: it does not assign an address-indexed causal identity to each physical copy. Licenses are not runtime data supplied to the selector or decoder. The preservation proof follows actual constructors: canonical deletion, frame and route steps, appender responses, rebuilt Local ancestors, clock/fuel phases, jobs and complete scheduler stages. Copying an already licensed subtree preserves its license; a newly

completed response receives the label of the consumed CTS prestate and the literal post-deletion carrier as its audit. Registered ancestor reconstruction preserves the fields that identify these licenses.

Lemma 8.1 (Generated origin invariant). *In every generated scheduler prefix whose source CTS prestates are nonempty, each fresh completed Local occurrence has a licensed label/snapshot pair from the actually executed source-response prefix. Its frozen audit equals the associated literal deleted carrier. No selected label can first appear before a response with that source label completes.*

Proof structure. The local induction is proved for the actual mutation constructors, rather than an abstract closure assumption on all possible terms. `SOnlyProvenance`, `SOnlyGeneratedOrigins` and `SOnlyCanonicalOrigins` establish local preservation, including deletion and copied fields. `SOnlySchedulerOrigins` and the global ancestor, terminal and stage modules lift this invariant through the concrete scheduler. Exact mutation chains list post-contraction configurations in order. Induction on their constructors equates list position j with the corresponding global sample, and concatenation preserves that chronology. The resulting global invariant has no caller-supplied origin annotation in the final theorem. $\square$

A stage of horizon h runs jobs that restart from the original bitword and execute source responses $0, \dots, h - 1$ in order. This repeated-prefix schedule is part of the imported construction. Source nonemptiness up to the first designated event, established by the compiler, is sufficient for the first-event argument. For nonhalting compiled inputs the source remains nonempty forever.

8.2 Sharp creation time and a common audit

Let r be the first CTS prestate whose phase is 17 and whose head bit is true. Earlier executed responses have different labels. Thus Lemma 8.1 excludes an F_{17} match in every earlier job and in the prefix before the first response to r . The exact response trace then resolves the remaining interior samples.

Lemma 8.2 (First acceptance and witness agreement). *For a compiled bitword b with first designated CTS event r , the first accepted S sample is*

$$\tau(b, r) = \text{firstJobPreResponseTime}(b, r) + 57.$$

Every occurrence matching F_{17} in that tree has the same frozen audit at relative address LLLR. That audit publicly decodes to the tail of the consumed CTS word.

Proof structure. One selected C4 contraction precedes the 56-contraction response. In the response's earlier positions, the new shell is not yet the completed pattern; previous licenses still exclude the designated label. Its final contraction creates the first complete shell with that label. The response-position lemmas are lifted to the whole tree, including copies and reconstructed ancestors. They show that every match in the first accepted sample has the same creation snapshot, not merely that one convenient occurrence is correct. The public seed-free carrier decoder agrees with the semantic decoder on this snapshot. The checked result is `SOnlyGlobalFirstEvent.firstSourceEvent_current_read`, with structural witness agreement supplied by the first-event and global-origin modules. $\square$

The offset 57 is a native-contraction offset in this exact pinned response, not a general bound on simulation time. Scheduler time before the response can be much larger.

8.3 A decoder on the current tree

The fixed decoder searches in root-left-right preorder and retains the first matching shell. It does not skip a malformed numerical payload in favor of a later witness. It extracts the LLLR audit, applies the public carrier grammar, and restores the deleted true bit. At the designated phase the recovered bitword starts with 10 and then complete 19-bit one-hot blocks. Restoring the known selected symbol 18 yields the UT19 event word.

The numerical reader finds the first maximal run of symbol 16. For a halting BP2 counter value x , its length is 4^{x+1} . The reader tests the power-of-four form and recovers x . The register compiler's output convention is $x = 2v + 3$, so the fixed final arithmetic $(x - 3)/2$ returns register result v . The reader rejects malformed lengths or an invalid odd-value convention. It never constructs 4^x to search for a match, reruns the source, or searches a trace. Lemma 8.2 makes the first-preorder policy sound even when the first accepted tree has multiple matching occurrences.

8.4 Completion of the main proofs

Proof of Theorems 3.1 and 3.2. The finite-register compiler preserves and reflects halting and gives the literal BP2 result $2v + 3$. The assembled UT19 simulation preserves and reflects BP2 halting; its nonhalting proof covers every microstep, and its first event has the stated counter-run grammar. One-hot simulation identifies the first CTS event and excludes earlier exhaustion. Proposition 7.1 provides the actual native-S trajectory; Lemmas 8.1 and 8.2 identify its earliest accepted sample and every witness payload. The fixed numerical reader therefore returns v there.

For the converse, suppose no designated CTS event occurs. The compiler's nonexhaustion certificate then makes every CTS prestate nonempty. The global stage invariant therefore applies to every finite S prefix and excludes its event pattern. Contraposition gives a CTS event from an accepted S sample, hence a source halt by the two source simulations. Halting absorption and determinism give a unique register result. Equality of first acceptance indices then yields the value-specific equivalence in Theorem 3.1. This is the actual composition in `SOnlyUniversality`; source liveness and numerical-success premises of component lemmas have been discharged by the compiled source.

The concrete tape-to-three-register compiler preserves every finite run and every represented tape cell. Positive macro durations make its checkpoints cofinal; an internal register halt is reflected to a source halt by absorption. Composing its `input_halting_iff` with the register result proves Theorem 3.2. Its register 0 is exactly the specified finite left-stack code, giving the stated TM-specific numerical result. $\square$

9 Effective bounds

9.1 Encoding

For R source registers and Q labels, put $k = R + 5Q$, $N_c = 4k + 1$ and $c = 8k + 3$. If s is the sum of the natural inputs and L is the literal BP2 program length, the actual uniform-helper compiler has the checked bound

$$L \leq 2c(2s + 9) + N_c(18c + 2) + c + 4N_c + 4.$$

It chooses memory depth $d = \lfloor \log_2(L + 2) \rfloor + 2$, so the width satisfies $W = 2^d \leq 4L + 8$. The compact initializer and all initial counters have size polynomial in c, L, W ; large source values are initialized by the compiled program's execution, rather than by inserting

their exponentially expanded counter representation into the seed. With fixed P and A , S generation has size and construction cost linear in the bitword, up to fixed code constants.

For an explicit binary description of length b , source dimensions are bounded by the tabulated description size and s is at most singly exponential in b . Consequently the complete syntax encoder has a bound $2^{\text{poly}(b)}$. The tape compiler has states·symbols·(5(symbols+1)+5)+1 labels. An n -symbol stack code is at most (symbols+1) ^{n} −1, so its binary length is $O(n \log(\text{symbols}+1))$. These facts preserve a singly exponential bound for the conventional-tape encoder as well. This is a source-description bound, independent of whether or when the source halts.

9.2 Observation and readout

The fixed bottom-up observer of Lemma 7.2 takes linear time in occurrence-tree size, with a construction-dependent constant. For a decoder input of size m , witness search visits at most m occurrences and uses a fixed pattern at each node. The selected audit is a subterm.

The carrier decoder has at most one recursive continuation per layer, always on a strict descendant, and reaches at most one base cell-spine decoder. Thus there are at most m charged carrier/cell iterations and at most m decoded bits. A classifier may compare arbitrary continuation subtrees, and the append-based spine-argument function can take quadratic work on a malformed long spine. A fixed quadratic allowance per iteration gives a safe $O((m+1)^3)$ constructor/list bound. The remaining fixed-block parser and numerical tests have polynomial bit cost; dynamic numerical counts are bounded by the decoded word length. This establishes a polynomial all-input algorithm, including malformed terms and rejected payloads.

The proof deliberately separates two levels. `SOnlyDecoderBound` checks strict size decrease, output-length bounds, fuel equivalence, charged recursion and quadratic spine visits. `SOnlyDecoderBoundCurrent.readFuel_eq` binds the bounded copy to the actual first-preorder decoder on every input. The remaining local-parser operation estimates and standard bit-cost interpretation are written arguments. The cubic statement is not a fully instrumented Lean runtime theorem. Upstream’s different row/terminal/bit readers already have a fully instrumented quartic certificate [6]; comparing their exponents requires aligning both functions and cost models.

10 Formal artifact and reproducibility

The development uses official Lean 4.33.1 [11] and the generic upstream source at commit 85a867988442fc423279341200f81634a1e65582. There are 67 project modules, including eleven additions for the conventional-tape bridge and its checks, and a 423-module pinned upstream dependency closure. No claim is made that this dependency replay covers the upstream release’s complete 1,239-module headline package.

A clean build of all project modules and an import-only wrapper completed successfully. Official `leanchecker --fresh` replay then accepted the aggregate. A separate audit queried 1,359 explicit public declarations and types; dependencies were restricted to `propext`, `Quot.sound` and `Classical.choice`. The encoder, finite-label compiler, observer and current-tree decoder have no choice dependency in their data cones. No new axiom, unfinished proof or native-decision shortcut is used.

The portable helper independently rebuilt all 490 upstream/project source modules plus its wrapper in a fresh isolated workspace with no old project or dependency objects mounted, and then passed fresh kernel replay. It verifies pinned source/configuration hashes before

building, refuses a nonempty output directory, executes no Lake configuration, and uses only the selected official library and fresh outputs. Recorded observations are:

Check	Wall seconds	Peak sampled RSS
67 project modules and wrapper	57.170	1.135 GB
Aggregate fresh replay	125.013	1.475 GB
Full portable source build and replay	633.802	1.516 GB

These are measurements of proof checking, not of simulated source computation. The portable run’s replay portion took 136.142 seconds. Independent rehashing covered 426 pinned source/config files, 424 dependency outputs, all 17,505 installed toolchain files, and 136 final source/output/wrapper files. The isolated proof runs used read-only proof inputs and denied socket creation.

Positive and negative controls used the same unchanged checker. A valid proof passed, an ordinary invalid proof of False failed source checking, and an explicitly injected invalid theorem was rejected by fresh replay for a kernel type mismatch. Canary objects were excluded from the proof search path. Fresh replay is an additional pass through the official kernel, not an independently bootstrapped alternate kernel.

Independent semantic review examined the source compiler, all-microstep halt reflection, first-event composition and exact output scope. Bounded transcriptions checked 29,400 assembled transitions and 135,048 tape/stack steps, among other cases. All 721 current Python test methods passed across a recorded completed prefix and resumed suffix; the earlier complete 693-method baseline also passed. These tests detect transcription defects and support reproduction; they do not replace the quantified formal statements.

The source packet includes the pinned upstream sources and MIT notice, exact module identities, theorem types, declaration dependencies, command records, all relevant logs, and the portable build helper. The principal result manifest is `results/formal_turing_universality.json`; the register result remains `results/formal_universality.json`. The main source identity is recorded in Appendix A.

11 Scope and further questions

The result concerns one explicitly prescribed finite-controller selected path. It establishes neither strategy-independent simulation nor universal computation through S normal-form termination. The finite control, root reset, local observations, read-only navigation, native-only mutations and invocation bound expose exactly what the reduction strategy contributes. A bounded next-step routine alone would not settle every possible interpretation of auxiliary computational power; the stated controller model is part of the theorem.

The compact phase count provides a concrete alternative instantiation, not a minimality theorem or a measured global speedup. Sharper controller-state and native-contraction comparisons, a fully instrumented cost semantics for the current numerical reader, and a canonical tape-output normalization theorem are separate potential extensions. No unproved conjecture is used in the construction.

The S-combinator challenge announcement expressly discusses selected paths and observation during nonterminating evolution [12]. Whether a particular construction satisfies the committee’s intended admissibility and originality criteria is an adjudication question, not a theorem of the formal model. The contribution and attribution above are mathematical claims; this paper asserts no prize award or priority over the earlier public universality construction.

A Theorem entry points and reproduction

The shortest semantic entry points are:

- `SOnlyTuringUniversality.halting_iff_event`: arbitrary finite-alphabet conventional tape source and finite input, with the fixed S event.
- `SOnlyTuringUniversality.selector_executes` and `every_step_native`: the actual chosen sequence and native steps.
- `SOnlyUniversality.result_iff_first_event`: exact arbitrary-register numerical output at the first accepted tree.
- `SOnlyTuringUniversality.left_stack_result_iff_first_event`: the explicit finite-left-stack convention.
- `SOnlyGlobalFirstEvent.firstSourceEvent_current_read`: sharp event creation and common first-event audit.
- `SOnlyTapeCompiler.input_halting_iff`: conventional tape source to the concrete three-register machine.

With Python 3.10 or newer, the official Lean 4.33.1 distribution and the included pinned dependency source directory, the reproducible command is:

```
python tools/replay_formal.py --target turing \  
--lean-bin /absolute/path/to/lean-4.33.1-linux/bin \  
--upstream-source /absolute/path/to/upstream \  
--build-dir /absolute/path/to/new-empty-replay
```

The optional `--check-only` verifies identities and dependency order but performs no proof check. The helper is a direct offline build procedure, not itself an OS sandbox. Its selected source manifests are pinned by hash. It does not automatically repeat the separately recorded declaration audits or negative controls.

The main theorem source `formal/SOnlyTuringUniversality.lean` has SHA-256

9a8b1e653f5e5f0c206a4be305deb24be51e7e1f7ae93e9e9a200862d02056a3.

The 67-module result manifest has SHA-256

588778633d2b5a26288a4b77ed870dd2730a282e0ff67fadfd64bcb4a913916a.

The recorded official Linux x86-64 Lean distribution archive has SHA-256

0376ac87487246b40dd077268c097e701f552e94c6d020d2373b50c7444fa22f.

References

- [1] Cinematic Strawberry. *Pure S Is Computationally Universal Under a Fixed Root-Restarted Finite Controller*. Manuscript dated 14 September 2026; Alexander Filin, project coordinator. Pinned repository revision 85a867988442fc423279341200f81634a1e65582. Manuscript and supporting MIT-licensed formalization.
- [2] Cinematic Strawberry. `PureSFormal.RootResetHeadline` and `PureSFormal.RootResetChallenge`. In the formalization of [1]. Pinned headline source.

- [3] Cinematic Strawberry. `DeterministicTapeInfiniteTape`. Conventional integer-indexed Boolean tape bridge in [1]. Pinned source.
- [4] Cinematic Strawberry. `DeterministicTapeRootResetOutput` and `RootResetRegularMarkerLanguage`. Exact current-tree observations and regular halting in [1]. Output source; regular-language source.
- [5] Cinematic Strawberry. `RootResetFiniteAllInputsTraceAgreement`. Generic finite-CTS realization in [1]. Pinned source.
- [6] Cinematic Strawberry. `DeterministicTapeOutputQuartic`. Instrumented all-input output resource certificate in [1]. Pinned source.
- [7] ais523. *UT19*. Esolang Wiki, revision 154545. CC0 production table and source construction. <https://esolangs.org/w/index.php?title=UT19&oldid=154545>.
- [8] ais523. *Brainpocalypse II*. Esolang Wiki, revision 172507. https://esolangs.org/w/index.php?title=Brainpocalypse_II&oldid=172507.
- [9] ais523. *The Waterfall Model*. Esolang Wiki, revision 193038; the account credits zzo38's independent earlier model. https://esolangs.org/w/index.php?title=The_Waterfall_Model&oldid=193038.
- [10] ais523. *The Amnesiac From Minsk*. Esolang Wiki, revision 173298. https://esolangs.org/w/index.php?title=The_Amnesiac_From_Minsk&oldid=173298.
- [11] Lean FRO. *Lean 4.33.1*. Official toolchain release. <https://github.com/leanprover/lean4/releases/tag/v4.33.1>.
- [12] Stephen Wolfram. *1920–2020 and a \$20,000 Prize: Announcing the S Combinator Challenge*. 8 June 2021. Official announcement.